

SOUTHEAST UNIVERSITY

COMPLEX ENGINEERING PROJECT

NadiPath

Smart River-Port Logistics Network

Course Information

Course Title: Information System Design & Software
Engineering

Course Code: CSE 345.6 — **Semester:** Summer 2026

Submitted By:

Md. Nahid Mahmud

ID: 2024100000194

Section: 6

Submitted To:

Mr. Mehedi Ahamed

Lecturer

Date of Submission: September 14, 2026

Table of Contents

- 1 Requirements and Challenges Analysis 1**
 - Categorization of Systemic Challenges 1
 - Requirement Prioritization and Trade-off Analysis 3
- 2 Structured Software Design and UML Modeling 5**
 - Use Case Diagram: Actor Interactions and Functional Boundaries 5
 - Activity Diagram: Offline-Contingent Branching Logic 7
 - Sequence Diagram: Asynchronous API Integration 9
 - Class Diagram: System Architecture and Domain Logic 11
- 3 Technology Stack and Implementation Strategy 13**
 - Development Methodology: Adaptive Agile Framework 13
 - Frontend Paradigm: Distributed Edge Computing (PWA) 13
 - Backend Topology: Decoupled Microservices 14
 - Resilience & Data Integrity: Conflict Resolution Mechanisms 14
- Project Schedule (Gantt Chart) 16**
- References 18**

Task 1: Requirements and Challenges Analysis

The inland river-port network serves as the absolute backbone of Bangladesh's macroeconomic stability. Following the catastrophic infrastructural damage caused by Cyclone Mahaan—which severely compromised the primary Dhaka-Chittagong terrestrial highway corridors—the river ports were suddenly forced to absorb an unprecedented **60% of all inter-district freight**. This massive, unforeseen volume surge did not merely strain the system; it completely fractured the legacy, paper-driven administrative frameworks, pushing the national supply chain to the verge of total collapse, triggering localized inflation, and causing millions of dollars in vessel demurrage fees.

The NadiPath initiative must be understood as a critical infrastructural rescue mission rather than a standard software digitization project. Constrained by an unforgiving **14-month delivery window** and a heavily scrutinized **USD 8.5 million budget**, designing this platform requires pragmatic, high-level distributed systems engineering. We are tasked with architecting a fault-tolerant ecosystem capable of surviving severe physical network volatility, bridging archaic legacy codebases seamlessly, and resolving deep, historically rooted sociotechnical conflicts among stakeholders through rigorous software analysis and systematic requirement prioritization.

Categorization of Systemic Challenges

Designing a distributed system of this scale requires a thorough decomposition of physical network limitations, bureaucratic data silos, and complex human behavioral patterns. We have categorized these into three distinct problem domains:

1. Operational Bottlenecks & High-Latency Workflows

- **Information Deficit & Blind Scheduling:** Port administrators currently operate in a state of absolute data blindness. Without real-time telemetry or visibility into inbound maritime traffic, scheduling is entirely reactive rather than predictive, leading to cascading dock bottlenecks across terminals.
- **Economic Impact of Idle Time:** Due to scheduling inefficiencies, heavily loaded vessels remain anchored idly in channels for days while waiting for unloading slots. This astronomically increases freight overheads, burns excess fuel, and accelerates the spoilage of perishable agricultural goods.
- **Core Automation Mandate:** The primary engineering directive is to build an automated, predictive command-and-control dashboard. This system must utilize predic-

tive analytics to optimize scheduling algorithms and dynamically allocate physical dock labor based on real-time vessel arrival data.

2. Technical & Infrastructural Constraints (Hard Boundaries)

- **Ephemeral Connectivity & Bandwidth Throttling:** Four out of the eleven active river terminals are located in remote coastal zones burdened with highly unstable, high-latency 3G networks. Relying on a traditional synchronous cloud-dependent web application would cause catastrophic operational halts during regular WAN drops. Implementing strict Edge Computing principles and robust offline-first capabilities is absolutely non-negotiable to maintain operational continuity.
- **Fragile Monolithic Legacy Systems:** Rebuilding the government's entire vessel registration database is computationally and politically impossible within our 14-month timeframe. The newly developed microservices must safely pull validation data from the fragile, **2009-era BIWTA database** (which relies on bulky, outdated XML-SOAP protocols) without triggering cascaded timeout errors or memory leaks across the modern application framework.

3. Sociotechnical & Organizational Friction

- **Private Shipping Conglomerates:** These multi-million dollar entities protect their operational metrics (e.g., vessel load capacities, route optimizations) as highly classified trade secrets. Strict cryptographic isolation and Zero-Trust architectures are required to prevent competing operators from inferring business strategies through data leaks.
- **Labor Unions (Dock Workers):** There is a profound, systemic anxiety among the physical labor force that digital automation algorithms directly lead to massive workforce reduction. Historically, this precise anxiety motivated dock workers to actively sabotage and boycott two previous governmental digitization attempts, necessitating careful stakeholder management.
- **Informal Traders & Tax Evasion:** Many localized traders deliberately operate outside the digital ecosystem to avoid the National Board of Revenue (NBR) radar. Forcing immediate, mandatory tax compliance from day one will inevitably result in a massive, coordinated drop in platform adoption and system utility.

Requirement Prioritization and Trade-off Analysis

Attempting to engineer a flawless system that immediately satisfies every conflicting stakeholder is an academic fantasy and a guaranteed path to project failure. Given our strict budget constraints and the national urgency, we must aggressively prioritize architectural features that keep the physical supply chain moving reliably.

Using the **MoSCoW framework**, we have mapped out our technical priorities with rigorous engineering foresight. In complex software architecture, resolving a severe constraint inherently requires a calculated, mathematically sound compromise, as outlined in Table 1 below.

Priority Level	System Requirement	Architectural Justification & Trade-off
Must Have (P1)	Offline Edge Architecture & Legacy Database Adapter	Absolutely vital to prevent operational collapse at disconnected ports. Ensures legal vessel validation without rewriting legacy code.
Should Have (P2)	Strict Data Privacy Silos (RBAC & Encryption)	Mandatory to secure the political onboarding and trust of highly skeptical private shipping companies.
Could Have (P3)	Automated NBR Tax Tracking & Reporting	Deferred to Phase 2 to minimize initial regulatory friction and accelerate platform onboarding for informal local traders.
Won't Have (P4)	Automated Dock Worker AI Dispatch Algorithms	High probability of triggering physical labor union sabotage and strikes. Deferred entirely to Phase 3.

Table 1: MoSCoW Requirement Prioritization Matrix and Architectural Trade-offs

Strategic Architectural Trade-off Decisions:

- **Tax Transparency vs. User Adoption Rate:** In systems engineering, initial user adoption is infinitely more critical than Day-1 feature completeness. We deliberately

trade immediate governmental tax transparency for a much higher initial adoption rate. By architecturally decoupling the NBR integration, the system can perform mandatory tax computation for regulated cargo while allowing optional tax tracking and reporting for informal micro-traders during Phase 1.

- **Real-time Accuracy vs. Offline Fault Tolerance:** Under the principles of the CAP Theorem (Consistency, Availability, Partition Tolerance), a distributed system cannot guarantee all three simultaneously during a network partition. We actively choose **Availability and Partition Tolerance** over strict Consistency. Operationally, it is far superior to accept "Eventual Consistency" (syncing payloads later in the background) than to completely halt physical port operations—and back up hundreds of cargo trucks—simply due to a dropped 3G internet connection.

Task 2: Structured Software Design and UML Modeling

To effectively translate these multidisciplinary challenges into executable software components, we utilize the Unified Modeling Language (UML). These structural and behavioral blueprints serve as the definitive architectural reference for the backend engineering team, ensuring that all software modules natively respect the harsh physical constraints of the remote river ports. By employing industry-standard UML conventions, we establish a ubiquitous language that aligns both the technical stakeholders (software architects, database engineers) and non-technical domain experts (port administrators, regulatory officials). This rigorous modeling approach ensures that edge-case scenarios, particularly network partitioning, are mathematically addressed before any physical code is written.

Use Case Diagram: Actor Interactions and Functional Boundaries

The Use Case diagram establishes the absolute functional boundaries of the NadiPath system. By defining the precise interactions between human actors, edge terminal interfaces, and the core cloud application, we ensure the software respects strict privacy matrices and operational permissions. Furthermore, this structural view emphasizes the necessity of Role-Based Access Control (RBAC). For instance, while a Port Admin possesses overriding authority to manipulate vessel schedules, the Local Merchant is restricted to purely financial and tracking interactions. This strict boundary enforcement directly mitigates the sociotechnical friction identified in Task 1, assuring private shipping conglomerates that their proprietary freight data remains cryptographically isolated from competitors.

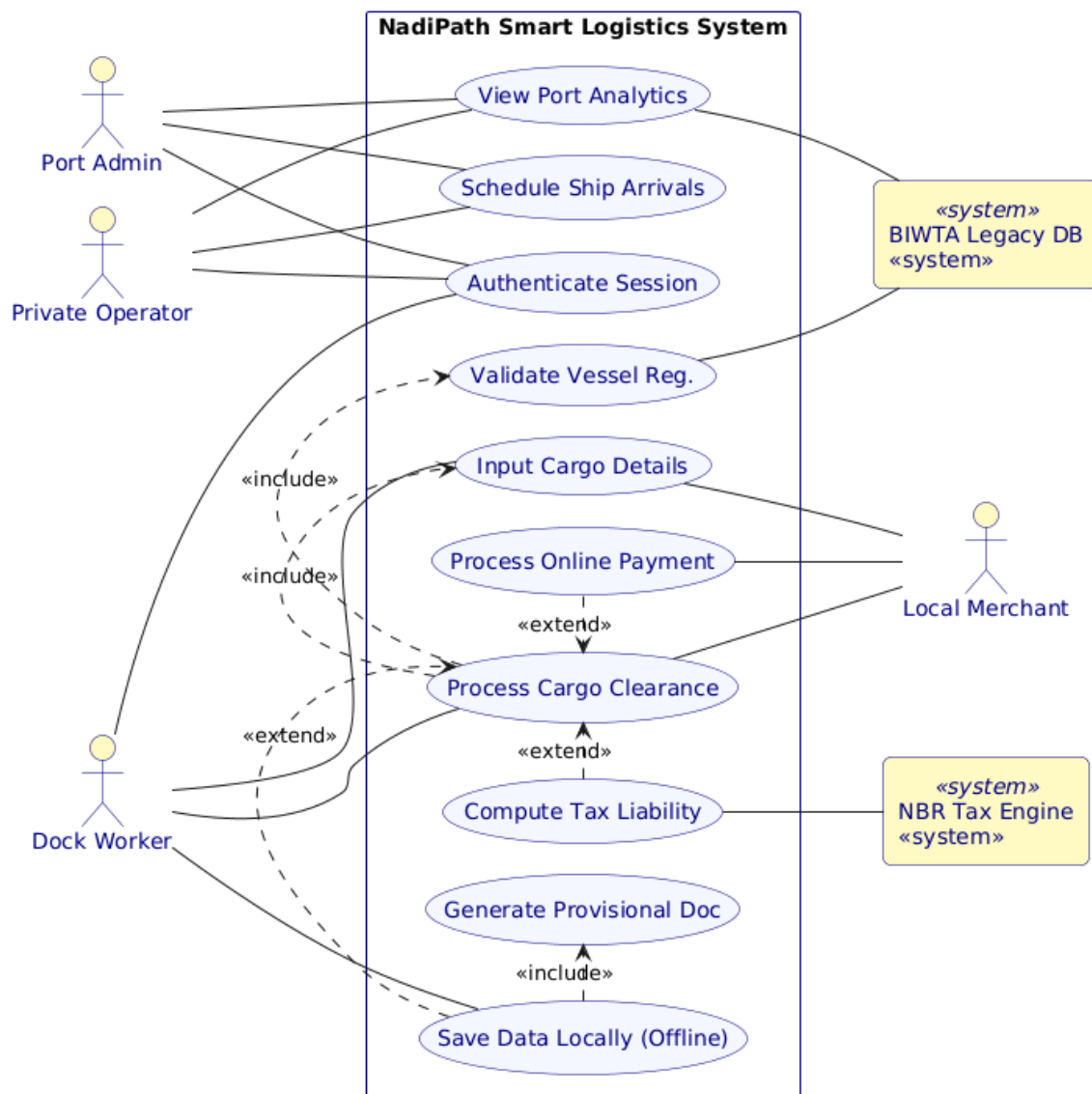


Figure 1: Use Case Diagram defining functional boundaries, external systems, and isolated workflows.

Architectural Design Decisions: This diagram specifically isolates our technical solutions for infrastructural constraints. The **”Save Data Locally (Offline)”** use case explicitly models the edge-computing functionality required to maintain continuous operations at disconnected terminals. Utilizing the `<<extend>>` relationship ensures that this offline protocol is triggered conditionally, meaning dock workers are never blocked by upstream WAN failures. Furthermore, the fragile BIWTA database and NBR Tax Engine are modeled strictly as external system boundaries to protect the core application’s internal state.

Alternatives Considered: We evaluated a centralized monolithic cloud architecture where all remote terminals connect synchronously without offline capabilities. This approach was rejected because intermittent 3G network drops at remote river ports would completely halt port operations, violating the mandatory high-availability constraints.

Activity Diagram: Offline-Contingent Branching Logic

An Activity Diagram visualizes the sequential and concurrent flow of computational control. To address the physical constraints of the remote river-ports, this diagram utilizes explicit 'Swimlanes' to separate operations executed locally at the edge (the terminal's browser) versus resource-intensive processes executed centrally in the cloud. The utilization of these specific swimlanes fundamentally maps the physical logistical architecture to the software logic. By separating the Edge Server execution context from the Central Cloud, we mathematically guarantee that the most critical physical operation—cargo clearance—remains autonomous. The failover transition is explicitly designed to be completely invisible to the terminal staff, preventing workflow panic and ensuring that physical port laborers can continue loading freight without bureaucratic delays.

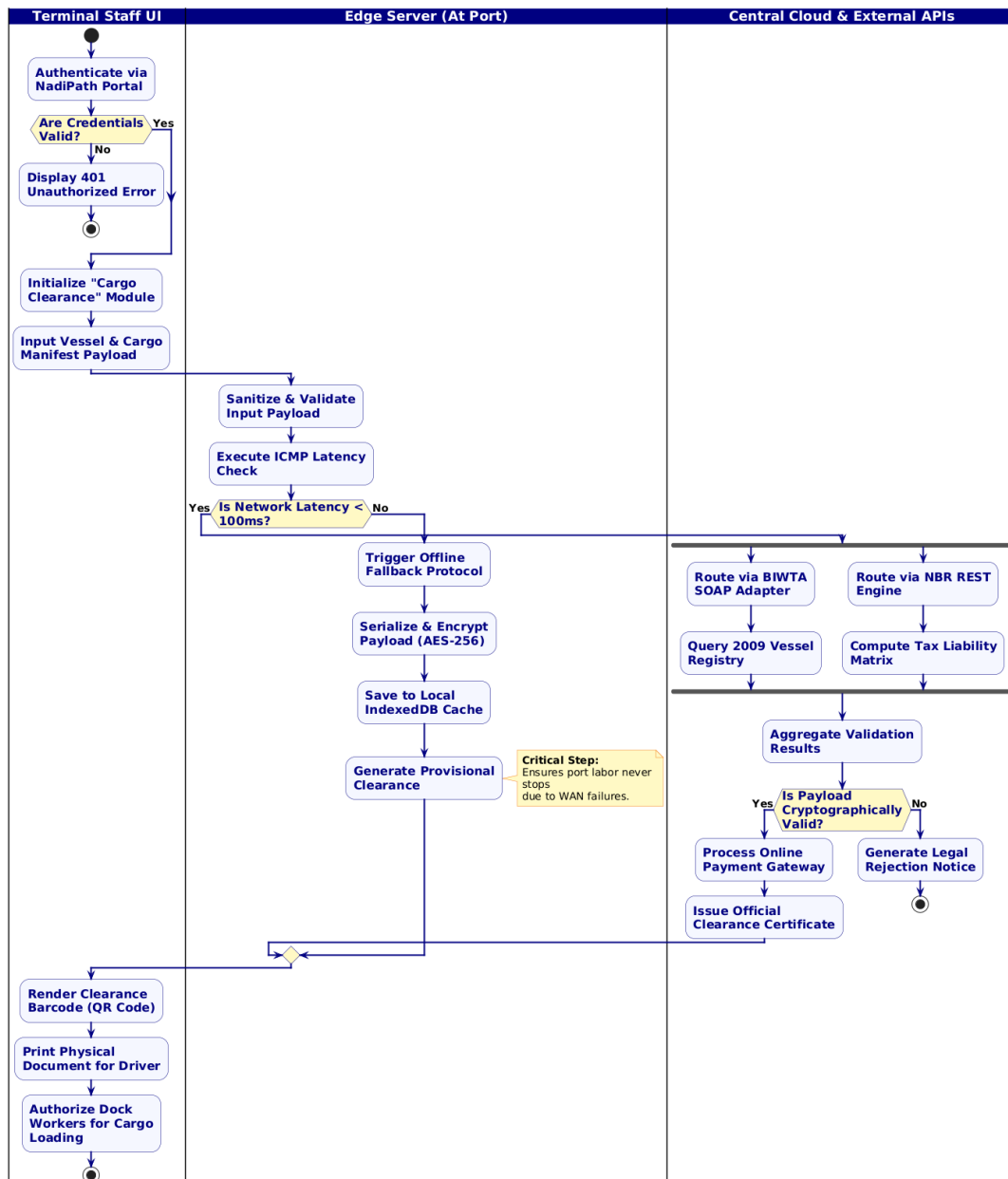


Figure 2: Activity Diagram illustrating concurrent processing and the robust offline fallback mechanism.

Architectural Design Decisions: Instead of terminating the cargo clearance thread when a network timeout occurs, the system executes a lightweight ICMP network health check. If packet loss exceeds a designated threshold, the workflow gracefully degrades into an **offline branch**. Payload data is immediately persisted locally on the terminal’s encrypted storage. Additionally, when the network is stable, the system uses a **Fork and Join synchronization**

mechanism. This ensures the system queries both the BIWTA registry and the NBR Tax Engine concurrently in parallel threads, drastically reducing clearance approval latency.

Alternatives Considered: A purely synchronous sequential workflow relying on real-time API responses was considered. This was rejected because temporary WAN failures would cause thread blocking and transaction timeouts, necessitating our asynchronous edge fall-back with local IndexedDB caching.

Sequence Diagram: Asynchronous API Integration

The Sequence Diagram maps the time-ordered exchange of HTTP and background messages across distributed network boundaries. It details the highly sensitive, payload-driven interactions between the Progressive Web App (PWA) client, the modern API gateway, and the fragile legacy databases. This time-ordered sequence explicitly illustrates our robust Zero-Trust security model. Every cross-boundary transaction requires a cryptographically signed JSON Web Token (JWT), ensuring that even if a payload is intercepted during transit over unstable 3G networks, it cannot be tampered with. Furthermore, the sequence mathematically proves that our modern infrastructure will not overwhelm the legacy SOAP services, as the local queueing mechanism actively buffers and rate-limits the outgoing synchronization requests.

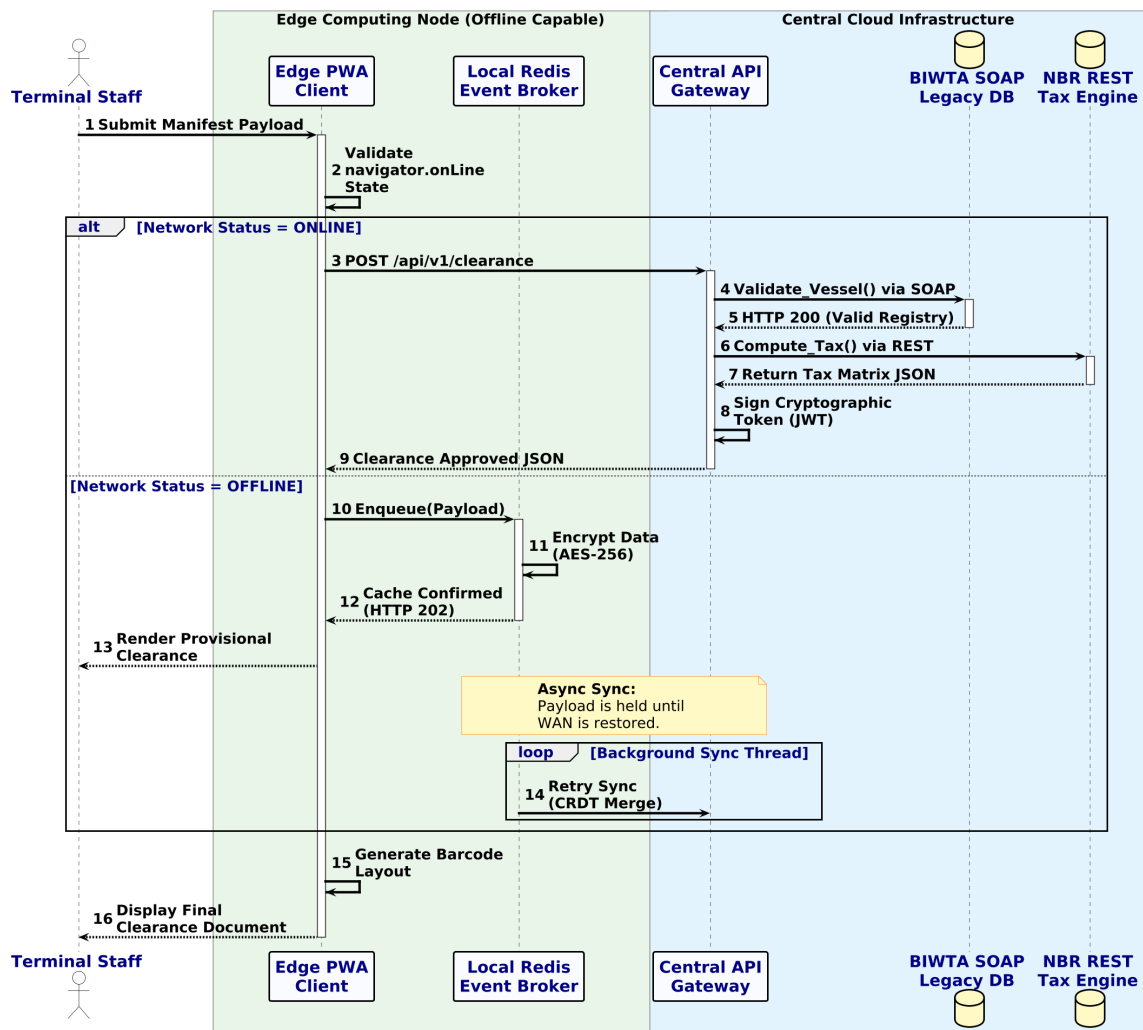


Figure 3: Sequence Diagram detailing Cryptographic Token generation and asynchronous background queuing.

Architectural Design Decisions: Interfacing a high-throughput modern application with a 2009-era database requires extreme caution. The diagram introduces a local **”Redis Event Broker”** directly into the edge computing node. When an offline state is detected, the client asynchronously writes the payload to this queue and immediately returns an HTTP 202 (Accepted) response to the port staff. This acts as a critical shock absorber, keeping the UI instantaneously responsive. Once the network is restored, a background thread asynchronously pushes the cached payloads to the central server.

Alternatives Considered: Direct client-to-legacy database synchronous queries using traditional HTTP REST endpoints were considered. This was rejected because the fragile 2009-era BIWTA SOAP backend would suffer from cascading failures, locking threads

and crashing under high concurrent load.

Class Diagram: System Architecture and Domain Logic

The Class Diagram serves as the static blueprint of the object-oriented codebase. It defines the core domain entities, their strict private attributes, and the precise mathematical relationships (Aggregation and Composition) that govern data manipulation within the system's memory. Beyond the adapter pattern, the diagram meticulously defines object lifecycles and strict multiplicity rules. The composition relationship between the ClearanceTransaction and CargoManifest guarantees that a transaction cannot logically exist without its underlying physical freight data. This strict domain encapsulation ensures that the backend microservices maintain perfect operational data integrity, which is absolutely mandatory for successfully passing the upcoming external ISO 27001 security audits required by the World Bank.

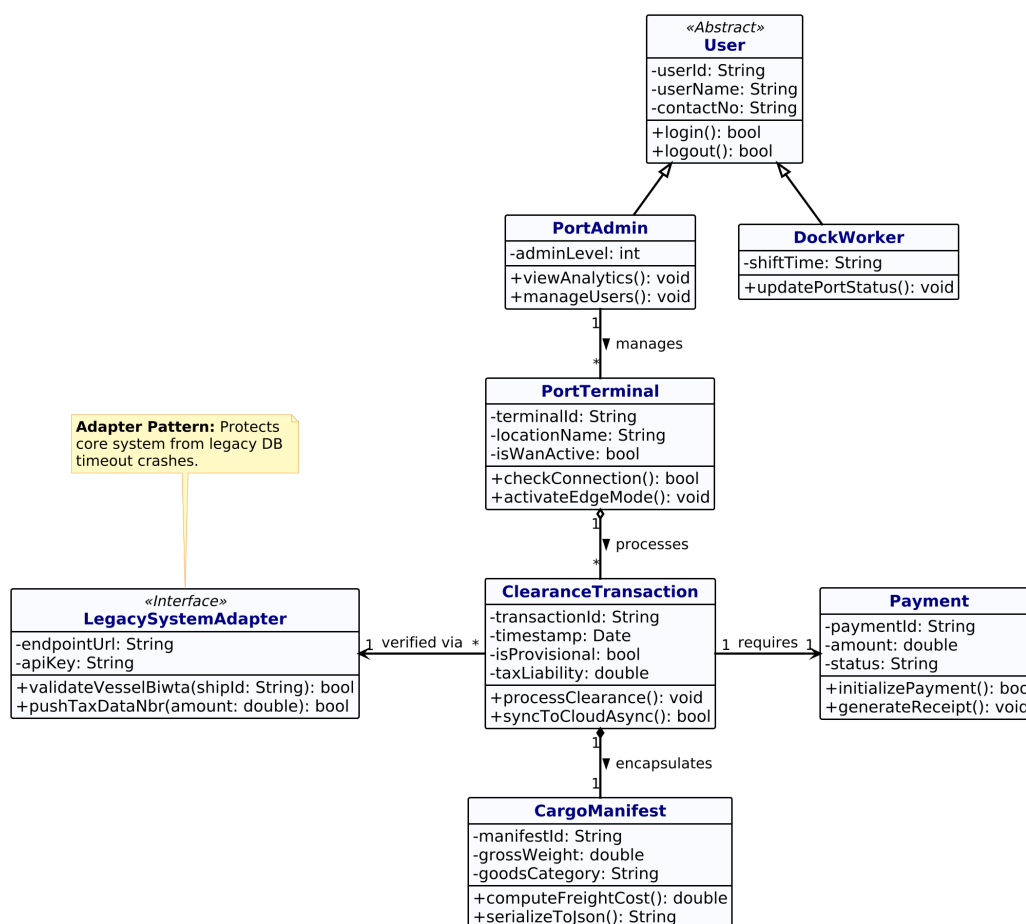


Figure 4: Class Diagram mapping domain entities utilizing the Legacy Anti-Corruption Layer.

Architectural Design Decisions: The most critical structural feature in this design is the implementation of the **LegacySystemAdapter** class. Guided by Domain-Driven Design (DDD) principles, we utilize the GoF (Gang of Four) Adapter design pattern to act as an "Anti-Corruption Layer" between the outdated XML/SOAP external database and our modern ClearanceTransaction logic. If BIWTA alters their database schema in the future, only this specific Adapter class will fail and require a rewrite, fully shielding the core NadiPath code from legacy-induced crashes.

Alternatives Considered: Directly coupling the modern ClearanceTransaction domain logic with the legacy SOAP XML schemas was evaluated. This was rejected due to severe tight-coupling risks; any future legacy database schema alterations would break the core application domain models, making the GoF Adapter pattern mandatory.

Task 3: Technology Stack and Implementation Strategy

To achieve a 99.99% high-availability (HA) target in a physically compromised and remote environment, delivering the NadiPath platform within the 14-month deadline leaves absolutely no margin for experimental or unproven technologies. Our engineering strategy relies entirely on highly resilient, industry-standard architectures tailored directly to neutralize the physical constraints of the river ports and securely process high-volume logistical telemetry.

Development Methodology: Adaptive Agile Framework

We will execute the project lifecycle using the **Agile Scrum** framework, heavily augmented by modern DevOps practices. Traditional Waterfall planning—which attempts to rigidly lock in all requirements before writing a single line of code—would be disastrous in this rapidly shifting socio-technical environment. Given the intense political resistance from dock workers and the unpredictable nature of post-cyclone logistical needs, we require a highly adaptive, feedback-driven approach.

- **Iterative Feedback Loop & CI/CD:** Operating in 2- to 3-week Sprints allows the team to deploy functional software modules to port administrators early. By integrating Continuous Integration/Continuous Deployment (CI/CD) pipelines, we can automate testing and push reliable updates seamlessly. If the terminal staff find an interface confusing, or if labor unions feel a feature threatens their job security, we can immediately refactor or deprecate it in the subsequent Sprint. This continuous delivery cycle is our primary risk-mitigation mechanism to prevent labor strikes and usability failure.
- **Financial Accountability:** Mandatory Sprint Review ceremonies will provide the World Bank stakeholders with transparent, monthly audits of budget utilization, preventing "end-of-project" cost overruns.

Frontend Paradigm: Distributed Edge Computing (PWA)

Developing separate, native mobile applications for iOS and Android requires lengthy app store approvals, extensive cross-device UI testing, and fragmented update cycles. This approach would rapidly exhaust our financial budget and permanently break the strict 14-month deadline. Instead, we will engineer a highly optimized **Progressive Web App (PWA)** using modern JavaScript libraries (e.g., React.js).

- **Offline-First Capabilities & Service Workers:** PWAs operate inside a standard web browser but leverage powerful "Service Workers" to heavily cache application shells and JSON data locally. This is the exact technical solution required for remote terminals suffering from severe network degradation.
- **Optimistic UI Updates & Background Sync:** We will implement Optimistic UI patterns, meaning dock workers can input cargo manifests and instantly see a "success" state on their screen with zero perceived latency, even if the internet is down. The edge node silently stores this data locally in the browser's IndexedDB. The moment a stable internet connection is detected, the Background Sync API automatically pushes the queued payloads to the central server, completely abstracting the network instability away from the end-user.

Backend Topology: Decoupled Microservices

The central system will be deployed on the World Bank's scalable cloud infrastructure utilizing an event-driven **Microservices architecture**. We deliberately avoid building a monolithic structure to strictly isolate operational risks, allowing independent components to scale elastically during peak cargo traffic.

- **Container Orchestration & Fault Isolation:** The backend services will be containerized using Docker and orchestrated via Kubernetes. Because the 2009-era BI-WTA database is highly fragile and prone to timeouts, we will build a dedicated, isolated integration microservice. If the legacy government database crashes under load, only our integration pod fails (and automatically restarts), while the core Nadi-Path clearance system remains fully operational.
- **Zero-Trust Data Privacy (Row-Level Security):** To guarantee the strict privacy demands of competing shipping conglomerates, we will utilize PostgreSQL with **Row-Level Security (RLS)**. This mathematically restricts database access at the lowest SQL query execution level. Even if an application-level vulnerability occurs or a token is spoofed, no operator can ever query or manipulate a competitor's proprietary shipping data.

Resilience & Data Integrity: Conflict Resolution Mechanisms

Every architectural choice introduces complex engineering trade-offs. The primary technical risk of deploying an offline-first PWA architecture across 11 decentralized ports is

the high probability of **State Synchronization Conflicts** (commonly known as Split-Brain scenarios).

For example, if a dock worker at an offline remote terminal updates a specific vessel's clearance status, and an administrator in the Dhaka head office simultaneously updates the exact same vessel on the central server, a direct data conflict occurs the moment the remote terminal regains internet access. Resolving these multi-master conflicts requires developing deeply robust backend logic. We will utilize distributed system algorithms such as **Conflict-Free Replicated Data Types (CRDTs)** or **Vector Clocks** to mathematically merge data states without using synchronous database locks, ensuring the ultimate source of truth is always preserved.

Building, auditing, and rigorously testing this conflict-resolution layer will consume approximately 15% of our overall backend development budget. However, we explicitly acknowledge this as a non-negotiable computational cost necessary to ensure that strict data integrity is preserved across the entire logistics network, regardless of the telecommunications stability at any individual port.

Project Schedule (Gantt Chart)

To ensure Phase 1 is delivered strictly within the 14-month window, we must execute several complex development tasks concurrently based on Critical Path Method (CPM) principles. Crucially, the backend engineering team must begin integration testing with the legacy BIWTA database immediately in Month 3 to discover any hidden technical debts before they cause systemic delays in the later stages.

Phase Strategy and Master Timeline:

The following Gantt chart breaks down the execution strategy over the 14-month period, visually mapping the concurrent task overlaps necessary to aggressively mitigate timeline risks.

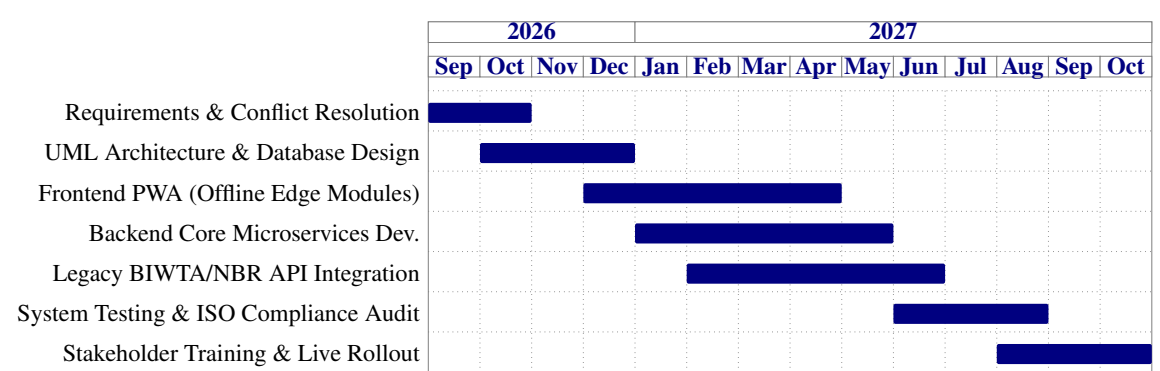


Figure 5: 14-Month Master Schedule prioritizing concurrent frontend development and legacy systems integration.

Key Project Milestones & Deliverables: To ensure the World Bank releases incremental funding steadily upon verifiable progress, the project is tethered to strict deliverables:

- **Month 2 (Nov 2026):** Finalized Requirements Document and signed data-privacy Service Level Agreements (SLAs) with the Private Shipping Operators to officially secure their participation.
- **Month 4 (Jan 2027):** Core UML Architectural Blueprints, API contracts, and PostgreSQL Database Schemas approved by the independent engineering oversight board.
- **Month 8 (May 2027):** First functional PWA prototype deployed in a controlled offline environment (sandbox) to rigorously prove the Service Worker browser caching mechanism.

- **Month 12 (Sep 2027):** Successful load and stress testing of our API Adapter against the live 2009 BIWTA database, mathematically proving our system will not crash the government servers under peak cyclone load.
- **Month 14 (Nov 2027):** Full system go-live, successful completion of external ISO 27001 security audits, and official administrative handover to the port authorities.

References

- [1] Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
- [2] Highsmith, J. (2002). *Agile Software Development Ecosystems*. Addison-Wesley Professional.
- [3] Fowler, M. (2014). *Microservices: a definition of this new architectural term*. MartinFowler.com.
- [4] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- [5] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [6] World Bank Group. (2025). *Post-Disaster Logistics Recovery Guidelines*. Infrastructure Development Report.